ReMoA: Reinforcement-Guided Mixture-of-Assessors for Domain-Adaptive Query Generation

Hao Wu[1], Jie Yang[2*], Xinguo Yu[1*], Sasha Nikolic[2], Rao Peng[3], Hao Ming[1]
[1]*Faculty of Artificial Intelligence in Education, Central China Normal University, Wuhan, China*
[2]*Faculty of Engineering and Information Sciences, University of Wollongong, Wollongong, Australia*
[3]*School of Computer, Wuhan Vocational College of Software and Engineering, Wuhan, China*
Email: {hwu, xgyu, hming}@ccnu.edu.cn, {jiey, sasha}@uow.edu.au, pr3250@whvcse.edu.cn

***Abstract*—In information retrieval, query generation primarily serves to improve the effectiveness of the standard two-stage pipeline, where a retriever first narrows the corpus and a re-ranker refines the order. Although recent query generation approaches have shown good performance, they still have practical limitations: they often depend on fixed domain priors, struggle to adapt under source–target shift, and lack a cooperative, multi-objective control that aligns generation with downstream retriever and re-ranking. We introduce *ReMoA*, the first Mixture-of-Assessors (MoA) framework tailored to query generation for re-ranking. *ReMoA* uses interpretable assessors to score candidate documents via complementary, retrieval-grounded priors, and employs a lightweight MoA router controlled by reinforcement learning to adaptively select informative and diverse documents. For each document, a large language model proposes multiple candidate queries; a dense retriever returns a normalized multi objective feedback vector. A covariance regularized policy gradient jointly updates assessor mixture weights and reward coefficients, aligning routing and generation with downstream retrieval while preserving exploration; semantic deduplication enforces non redundant, space covering query sets. Empirically on BEIR, *ReMoA* reduces the source–target gap and mitigates domain bias, delivering consistent improvements over strong query generation baselines with average gains of +1% nDCG@10 and +0.8% Recall@100 across target domains.**

***Index Terms*—Query Generation; Mixture of Assessors; Reinforcement Learning; Document Retrieval; Neural Rankers;**

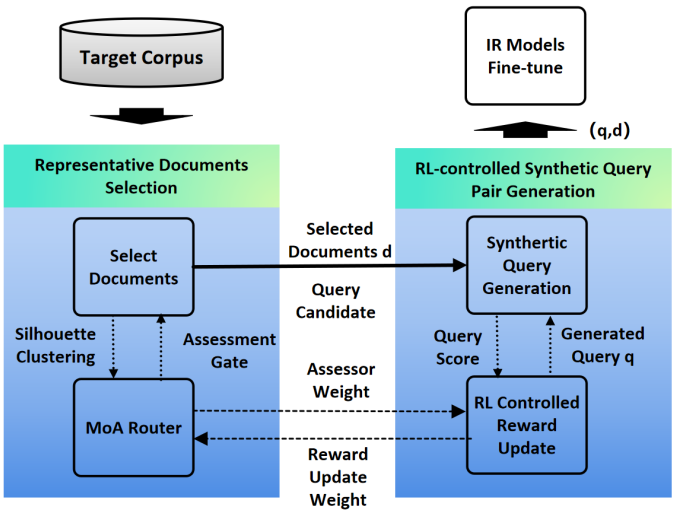


Fig. 1: The figure illustrates the *ReMoA*: document candidates are clustered and routed by a MoA to select documents; synthetic queries are generated; an RL-controlled reward module feeds back updated assessor weights to the router; and the resulting (q,d) pairs are used to fine-tune IR models.

I. Introduction

Modern document retrieval typically adopts a two-stage architecture: an initial dense retriever efficiently selects a candidate set from the corpus, followed by a more expressive re-ranker that refines the ordering with fine grained relevance scoring [1]. While recent advances in pretrained transformer models have substantially improved the performance of neural rankers [2], but neural retrieval and re-ranking systems trained on one distribution often face a significant performance drop when applied to a new target corpus due to distributional mismatch [1], [2], [3], [4]. A common remedy is to synthesize supervision in the target domain by generating query-document pairs for fine-tuning [5], [6], [7], [8]. In practice, issuing multiple high quality, complementary queries per document can substantially increase recall and coverage while preserving precision [9], [10], [11]. However, existing query generation pipelines still suffer from three main limitations. First, document selection is often based on fixed heuristics or static clustering, for example, using the full corpus, random subsampling, or choosing cluster-central documents as in DUQGen [8]. These strategies struggle to balance relevance, diversity, and long-tail coverage, often overlooking important but sparse topical pockets [12], [13], [14], [15]. Second, query generation usually optimizes a single proxy objective such as simple rank validation or templated prompts and under uses richer signals like discriminativeness, answerability, and set level coverage that matter for downstream ranking [9], [11], [16]. Third, Mixtures of Assessors (MoA) have improved retrievers and re-rankers [17], [18], [19], but have not been applied to the query generation stage; similarly, reinforcement

learning has been used to shape query generators [20], [21], yet prior work stops short of coordinating document selection with generation or exploiting multiple complementary assessors. These gaps limit the ability to create synthetic supervision that is simultaneously precise, diverse, and coverage seeking for re-ranking.

We address these limitations with *ReMoA*, a reinforcement driven MoA framework for query generation, specifically designed to improve re-ranking adaptation. *ReMoA* integrates assessor reasoning and reward driven control directly into the query generation pipeline. Concretely, each document is evaluated by a panel of interpretable assessors that capture complementary, retrieval relevant priors. These scores form the basis for a lightweight MoA router that dynamically weighs assessor opinions and selects documents for generation. Crucially, this routing policy is learned through multi-objective reinforcement learning, with the aim of surfacing under-covered and diverse content that static clustering often neglects. For each selected document, a Large Language Model (LLM) generates multiple query candidates, which are then executed by a dense retriever. Retrieval results are fed back as a normalized reward vector. Rather than optimizing for a single reward, *ReMoA* applies a covariance regularized policy gradient update that jointly adjusts the MoA assessor weights and reward preferences. This mechanism aligns routing decisions with downstream retrieval objectives while maintaining diversity among assessors and avoiding assessor collapse. Additionally, we employ a semantic deduplication module to filter redundant queries, and a rotation based clustering refresh strategy that incrementally reallocates focus toward under served regions of the corpus as training progresses. Together, these components form a tightly integrated control loop: document selection, query generation, and multi objective evaluation co-adapt through reinforcement feedback. As a result, *ReMoA* avoids brittle dependence on initial clusters, steadily improves long-tail coverage, and produces compact, diverse, and high-quality supervision that directly benefits re-ranking performance under domain shift.

The contributions of this paper are as follows:

- We present the first MoA framework for query generation in re-ranking adaptation, *ReMoA*. It introduces multiple assessors for evaluating documents and queries along with tailored retrieval-based reward signals, improving query diversity and coverage.
- We develop a collaborative MoA+RL control scheme that coordinates document selection and query generation. By exploiting complementary assessors, the controller adaptively favors representative yet diverse documents to guide query synthesis, overcoming the brittleness of fixed clustering heuristics.
- We demonstrate that the RL-guided MoA weighting produces queries that effectively fine-tune re-ranking models, reducing the source target domain gap and mitigating domain bias. The resulting queries consistently improve retrieval metrics.

II. Related Work

A. Domain Adaptation for Retrieval

Imformation retrieval typically follows a two-stage pipeline: (i) a first-stage retriever efficiently narrows the target corpus to a small set of candidates, and (ii) a second-stage re-ranker refines the ordering with a more expressive scoring function [1]. Transformer-based pretrained language models substantially improve both stages across diverse benchmarks [2]. In this setting, a neural ranker encodes queries and documents into dense representations to estimate relevance and return a ranked list. For example, dense retrievers such as ColBERT [22], Contriever [23], and GTR [24] provide first-stage candidates, which reordered via cross-encoder re-rankers like MonoBERT [25] and MonoT5-3B [26].

Despite these advances, performance degrades when the target corpus distribution diverges from that of the source corpus used to train the retriever, a well-known phenomenon referred to as domain shift, characterized by mismatches in vocabulary, document structure, or writing style between the two domains [27]. This challenge motivates domain adaptation, which aims to bridge the distributional gap between source and target corpora to maintain retrieval effectiveness under unseen or evolving domains.

B. Query Generation

A popular direction for addressing domain adaptation is **query generation**. The central idea is to synthesize informative queries for documents within the target corpus, thereby expanding supervision data without requiring manual annotations. Compared to traditional techniques such as **query expansion** [28], which augments an existing query with semantically related terms, and **query rewriting** [29], which reformulates a query to better capture user intent, query generation operates in a *zero-shot* setting. It produces entirely new pseudo queries that act as synthetic supervision signals, enabling the retriever to capture target-domain relevance patterns and effectively mitigate the impact of domain shift. In practice, a query generation pipeline typically involves three stages [8]: (1) **Document Selection.** Representative documents are sampled from the target corpus; (2) **Query Generation.** A generative language model produces candidate queries conditioned on selected documents; and (3) **Pair Filtering.** The generated query–document pairs are filtered to retain those with high semantic alignment and diversity.

a) Document Selection: UDL [12] and CCQGen [13], use the full set of target-domain documents. Others randomly sample p values from the corpus to reduce computation [14]. More selective methods aim to identify informative documents d in order to generate high quality queries q. DUQGen clusters the corpus and selects cluster-central documents to ensure diverse and representative sampling [8]. SynQGen applies normalized information content to avoid selecting trivial or noisy documents [15], while other work links semantically similar documents into joint documents to expand contextual expressiveness [20].

b) Query Generation: Template-based methods produce keyword style queries per topic [30], [31], while SynQGen are prompted with question triggers such as "what" or "how" to elicit simple interrogative queries [15]. Hyper-QKSG leverages document structure by pairing documents with extracted answers, using document titles as question prompts [16]. UDL link similar documents to simulate shared semantics [12], or model specific search intents to produce intent-aligned queries [32]. Simulate interactive query refinement via keyword concatenation or prompt fusion [30], [31]. Some methods generate both relevant and deliberately non-relevant queries for each document, either manually defined or derived from similarity-based contrastive groups [14], [20]. Coverage-based strategies estimate concept or phrase distributions from documents and iteratively steer generation toward underrepresented content, improving long-tail coverage and training diversity [13]. DUQGen uses few-shot prompting with (d, q) exemplars to condition generation [8].

c) Pair Filtering: SynQGen and CCQGen using BM25 or dense retriever rank positions to filter out low-quality queries whose original document does not rank within top-k [15], [13]. Rerankers like MonoT5 are often used to score (q, d) pairs, retaining those above a probabilistic threshold [30], [14]. More selective approaches further examine semantic validity or alignment. For instance, DUQGen and SynQGen directly reject queries that fail retriever validation or MonoT5 thresholds [8]. In contrast, contrastive feedback methods use model agreement or relevance ranking within similar document clusters as soft supervision signals to filter or optimize queries [20]. Some frameworks use multi-stage query validation. And multiple query variants per document using reciprocal rank fusion or blending methods to stabilize training signals from diverse generations [30]. Others incorporate QA-style answerability tests, LLM-based label consistency checks [14], or concept distribution alignment to enforce semantic fidelity to the document [13], [16].

Despite these advances, current query generation approaches often rely on predetermined heuristics for each stage of the pipeline. **Document selection** is typically performed only once, without iterative feedback to revisit previously unselected documents. **Query generation** often neglects adaptation feedback from the retriever. **Pair filtering**, when presents, usually applies static thresholds or similarity scores without dynamically assessing the quality of generated pairs. As a result, most existing pipelines operate in a one-shot manner, lacking mutual feedback between stages, and therefore fail to progressively align synthetic supervision with target-domain relevance patterns.

C. Mixture of Experts and Reinforcement Learning

Mixture of Experts (MoE) architectures are increasingly employed in document retrievers to enhance encoding and matching effectiveness. For example, CAME [17] implements a sparse retriever in which individual expert modules specialize in distinct query–document relevance patterns. Router-Retriever [18] constructs multiple domain-specific embedding models and learns a routing gate that dynamically selects the most suitable expert encoder at inference time. MoTE [33] integrates a mixture of task-specific experts within a shared embedding model to support multi-task retrieval learning. CartesianMoE [34] introduces a Cartesian-product routing mechanism to facilitate knowledge sharing across encoder layers. M2oERank [19] combines multiple ranking experts guided by reinforcement learning signals to refine the final relevance scoring. While these methods validate the effectiveness of expert routing on the retriever side, they have not yet been extended to the query generation task, leaving this direction largely unexplored.

Reinforcement learning (RL) has been widely explored in document retrieval to optimize ranking policies and embedding alignment. Typical applications include *reward-guided retriever training* [35], *policy-based reranking* [36]; and *interaction-aware optimisation* [37]. More recently, RL has been also applied to the query generation to align synthetic queries with downstream retrieval performance. Unlike fixed prompting or heuristic-based generation, RL introduces reward-driven optimization, enabling query generators to adapt dynamically through feedback from retrievers. However, current RL-based query generation methods often remain limited in scope. They typically apply reward optimization in a single pass without iterative feedback between generation and filtering, resulting in unstable supervision signals. Moreover, reward functions are often handcrafted or domain-dependent, constraining generalization to unseen domains.

III. Methodology

This section introduces a two-stage adaptive framework, **Re**inforcement-guided **M**ixture-**o**f-**A**ssessors (*ReMoA*) for query generation. In the first stage, multiple assessors independently assess each candidate document from different perspectives. Their outputs are aggregated through a gating mechanism that estimates the probability of a document being selected for pseudo-query generation. In the second stage, a Large Language Model (LLM) conditions on the selected documents to generate candidate pseudo queries, from which high-quality samples are further filtered for training. The entire process is jointly optimized under a reinforcement learning paradigm, where retrieval-based rewards dynamically adjust assessor routing and selection probabilities. As a result, the router progressively shifts its weighting toward documents that yield more informative queries while maintaining coverage over the entire corpus. Fig. 2 illustrates the proposed *ReMoA* method.

A. Task Definition

Let the target corpus be $D = \{d_i\}_{i=1}^{N}$ with N documents, where each document d_i has an embedding $\mathbf{e}_i$. A pretrained retriever R_{θ_0} underperforms on D due to distributional mismatch within the target corpus. The objective is then to adapt the retriever *using only D without labels* by synthesizing supervision via pseudo query–document pairs.

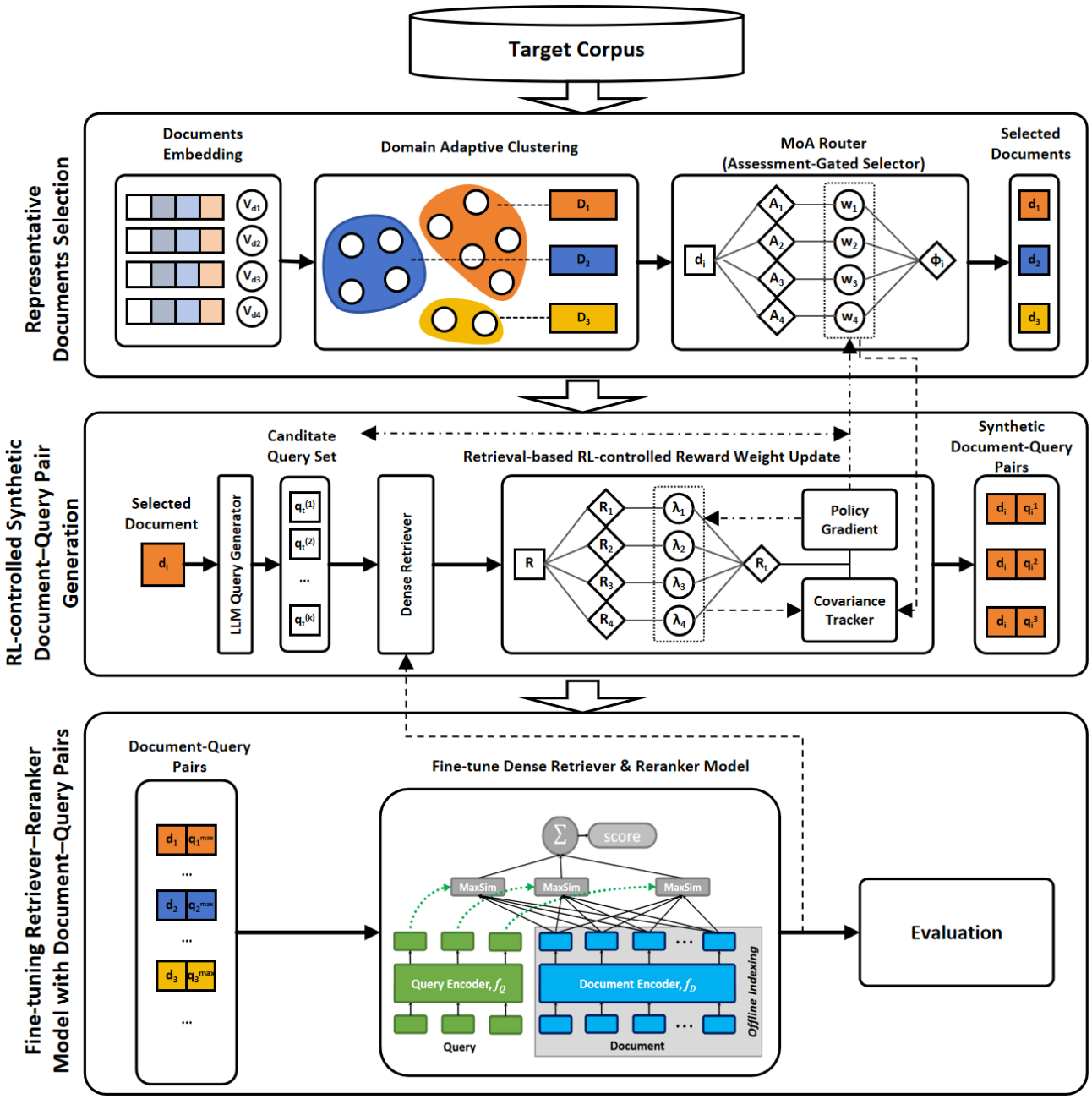


Fig. 2: Overview of *ReMoA*. The corpus is embedded and domain-adaptively clustered; multiple assessors score each document, and MoA router aggregates these signals to select a representative, diverse set. For each selected document p, an LLM proposes candidate queries that are evaluated by a dense retriever; retrieval-based rewards tracked via covariance and updated with policy gradients feed back to adjust routing and mixture weights, and high quality (d, q) pairs are retained. The synthetic pairs are then used to fine-tune a dense retriever and a late-interaction reranker, followed by evaluation.

Formally, we iteratively generate a set of T pseudo queries $Q = \{q_t\}_{t=1}^{T}$ and associate each q_t with one selected document $d_t \in D$, yielding a synthetic training set:

$$D_T \;=\; \{(q_t, d_t)\}_{t=1}^{T} \tag{1}$$

We then fine-tune the retriever from R_{θ_0} to R_θ on D_T with a ranking objective $\mathcal{L}(\theta; D_T)$ (e.g., contrastive or pairwise loss), so that the adapted model R_θ improves retrieval quality on D.

B. Mixture-of-Assessors

The first stage of our framework introduces a *Mixture-of-Assessors* (MoA) architecture that evaluates documents from multiple complementary perspectives and adaptively integrates these assessments to guide selection. Inspired by the expert specialization idea in *ReMoA* architectures, our MoA similarly

adopts adaptive routing across assessors, yet departs from representation learning to act as a document-level evaluator that prioritizes informative and representative documents for query generation. This stage comprises three key components: **corpus partitioning**, **heuristic assessors**, and **adaptive regularizer**.

1) Corpus partitioning: Given the target corpus $D = \{d_i\}_{i=1}^{N}$, we first establish a clustering structure that serves as a foundation for MoA-guided document selection. Partitioning D into semantically coherent clusters enables the router to *operate within smaller, more homogeneous subsets parallelly*, allowing assessors to make finer-grained relevance judgments. Moreover, it prevents the router from over-concentrating on dominant clusters by encouraging exploration across under-represented regions of the corpus.

a) Initialization: We determine the initial number of clusters through a silhouette-based analysis on the document embeddings $\{\mathbf{e}_i\}_{i=1}^{N}$ derived from D. That is, we compute the average silhouette coefficient $\mathrm{Sil}(k)$ for a set of candidate values $\mathcal{K}$ ($k \in \mathcal{K}$) and select the value yielding the most coherent structure:

$$k_{\text{init}} = \arg\max_{k \in \mathcal{K}} \mathrm{Sil}(k), \qquad \mathcal{C} = \{C_c\}_{c=1}^{k_{\text{init}}} \tag{2}$$

This partitions D into k_{init} clusters, represented by $\mathcal{C}$.

b) Rotation rule: Conventional approaches following cluster partitioning often adopt a global *top-k* selection strategy [8], which retrieves the highest-scoring k documents from each cluster. Although effective in ranking accuracy, this practice tends to over-represent dominant (head) clusters containing abundant documents while neglecting those in the long tail. To address this imbalance, we introduce a *rotation rule* that enforces balanced document selection across clusters under a quality constraint.

Specifically, let $\mathcal{C} = \{C_1, \ldots, C_K\}$ be the K clusters and $S(d_i)$ the score of document d_i. Given a target number T and a relative threshold $\tau \in (0, 1)$, at iteration t let U_t be the set of unselected documents and $\mathcal{S}_t$ the selected set. Choose the cluster with the largest current pool, $c = \arg\max_j |(C_j \cap U_t)|$. In $C_c \cap U_t$, let $d_c^\star = \arg\max_{d \in C_c \cap U_t} S(d)$. Select from $\{d \in C_c \cap U_t : \ S(d_c^\star) - S(d) \le \tau\, S(d_c^\star)\}$ up to the top-3 items (at least 1, at most 4 per visit), add them to $\mathcal{S}_t$. Move to the next cluster with the largest $|(C_j \cap U_t)|$ and repeat select until all K clusters are visited; keep rotating in rounds until $|\mathcal{S}_t| \ge T$. If $|\mathcal{S}_t| < T$. Recluster into K clusters refresh (Equation 3) and return to cluster. If still unmet afterward, prioritize the cluster with the largest $|C_j|$ among those with the fewest selections $|\mathcal{S}_t \cap C_j|$, and repeat select.

c) Cluster refresh: As query generation progresses, the unselected set U_t evolves and the initial partition $\mathcal{C}$ may become suboptimal for the remainder of the corpus. We therefore monitor the silhouette score of the current clustering restricted to U_t, denoted $\mathrm{Sil}_{U_t}(\mathcal{C})$, and trigger a refresh when topical coherence drops below a threshold τ_{sil},$U_t^c \leftarrow U_t \cap C_c,\ \forall\, c \in \{1, \ldots, K\}$:

$$\mathrm{Sil}_{U_t}(\mathcal{C}) < \tau_{\text{sil}} \ \Rightarrow\ \mathcal{C} \leftarrow \textsc{Recluster}(U_t) \tag{3}$$

This periodic refresh keeps the cluster structure aligned with the unselected portion of D, ensuring that the MoA-guided selection over clusters remains representative, diverse, and adaptive throughout the generation process.

2) Heuristic assessors: Following corpus partitioning, we introduce a set of heuristic assessors, quantifying document quality from diverse perspectives, that provide prior evaluation to complement the adaptive weighting in MoA. Formally, we define four heuristic assessors $\{s_m(i)\}_{m=1}^{M}$ for each document d_i, capturing density, representativeness, linguistic normality, and cluster centrality.

a) Similarity density $s_1(i)$: This assessor counts neighbors for each document d_i via similarities to cluster centroids. Let the current partition be $\mathcal{C} = \{C_1, \ldots, C_K\}$ with centroids $\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_K\}$. For a centroid–similarity threshold τ_{cent}, where $\mathbf{e}_i$ is the embedding of d_i. The neighborhood of d_i is then the union of cluster members that are also close to at least one of d_i hit centroids:

$$\mathcal{N}(i) \ = \ \big\{\, j \neq i \ \ i, j \in C_k,\ \cos(\mathbf{e}_j, \boldsymbol{c}_k) \ \ge \ \tau_{\text{cent}} \,\big\} \tag{4}$$

The similarity density is the normalized neighborhood size:

$$s_1(i) \ = \ \frac{|\mathcal{N}(i)|}{\max_j \ |\mathcal{N}(j)|} \ \in [0, 1] \tag{5}$$

A higher $s_1(i)$ indicates that d_i lies in a dense semantic region and is representative of a frequent topic cluster.

b) TF–IDF representativeness $s_2(i)$: We compute TF–IDFwithin each cluster, the score changes whenever the clustering changes. Let $\mathrm{tfidf}(t, i)$ denote the TF–IDF weight of term t from document d_i, and V the corpus vocabulary. The mean TF–IDF value is:

$$\mathrm{tfidf}_c(t, i) \ = \ \mathrm{tf}(t, i) \cdot \mathrm{idf}_c(t), \tag{6}$$

$$T_i^{c(i)} \ = \ \frac{1}{|V_{c(i)}|} \sum_{t \in V_{c(i)}} \mathrm{tfidf}_{\,c(i)}(t, i) \tag{7}$$

and with $T_{\max} = \max_{i \in V} T_i$, we normalize via a logistic sigmoid (*i.e.*, $\sigma(\cdot)$):

$$s_2(i) = \sigma\Big(\frac{T_i}{T_{\max}}\Big) \tag{8}$$

Because $\mathrm{idf}_c(t)$, V_c, and $c(i)$ all depend on the current clustering $\mathcal{C}$, any *cluster refresh* updates these quantities and thus changes $s_2(i)$.

c) Linguistic normality $s_3(i)$: This assessor measures linguistic fluency and naturalness. That is, let $\mathrm{NI}(i)$ denote the normality index of document d_i in C, computed as the per-token normalized negative log-likelihood under a pretrained language model [15]:

$$\mathrm{NI}(i) = -\frac{1}{|d_i| \log |V|} \sum_{t=1}^{|d_i|} \log P(w_t \mid w_{<t}) \tag{9}$$

where $|V|$ is the vocabulary size. Given corpus-level statistics μ_{NI} and σ_{NI}^2, we define:

$$s_3(i) = \exp\Big(-\ \frac{(\mathrm{NI}(i) - \mu_{\text{NI}})^2}{2\sigma_{\text{NI}}^2}\Big) \tag{10}$$

As such, this assessor rewards documents whose linguistic characteristics fall near the corpus norm, penalizing overly noisy text.

d) Clustering centrality $s_4(i)$: This assessor leverages the corpus partitioning introduced earlier to measure a document's positional importance within its semantic cluster. Let $\mathcal{C} = \{C_c\}_{c=1}^{K}$ be the set of clusters and $\bar{\mathbf{e}}_c = \frac{1}{|C_c|}\sum_{j\in C_c}\mathbf{e}_j$ the centroid of cluster C_c. For a document $d_i \in C_c$, define the cosine similarity to its cluster centroid as:

$$d(i) = \cos(\mathbf{e}_i, \bar{\mathbf{e}}_c) \tag{11}$$

and normalize within the cluster:

$$s_4(i) = \begin{cases} \dfrac{d(i) - \min_j d(j)}{\max_j d(j) - \min_j d(j)}, & |C_c| > 1, \\ \xi, & |C_c| = 1 \end{cases} \tag{12}$$

where ξ is a hyperparameter (set as 0.85 in our implementation). Consequently, documents that are closer to the cluster centroid achieve higher $s_4(i)$ values, reflecting stronger semantic centrality and representativeness within topical group.

Our approach is a cluster-update, rotation-based selection scheme. This design curbs head-cluster monopolies, strengthens long-tail coverage; while setup overhead is higher than some baselines, the consistent gains in quality, diversity, and stability compensate for the extra cost.

3) Adaptive regularizer: The MoA gating mechanism adaptively combines the outputs of multiple assessors to determine document selection probabilities. However, a common issue in mixture-based architectures is *assessor dominance*, when one assessor obtains a disproportionately large weight early in training and suppresses the contributions of others. To preserve diversity and ensure stable routing, two complementary regularizers are introduced hereafter.

Specifically, at the t-th iteration, let the current document-level assessor score vector $\mathbf{S}_{i_t} = [s_M(i_t)]^\top$ and the corresponding mixture weights $\mathbf{w} \in \Delta^4$, the router computes the weighted assessment score:

$$z_{i_t} = \mathbf{w}^\top \mathbf{S}_{i_t} \tag{13}$$

which determines the probability of selecting document d_{i_t}.

Then, we consider to prevent any single assessor from dominating too early, we regularize the weight vector $\mathbf{w}$ toward the uniform distribution. Practically, this is an entropy-style prior that encourages spread-out weights rather than peaky assignments, stabilizing training and improving coverage across assessors without prescribing a specific update rule. To ensure no assessor collapses to zero influence, we impose a minimum weight floor and renormalize on the simplex. In practice, this can be implemented by clipping each weight to at least minimum weight floor and then re-scaling so the weights sum to one. This acts as a regularizer not an update equation—guaranteeing persistent participation of all assessors while remaining lightweight to apply. These two steps are applied sequentially to maintain numerical stability, preventing dominance by a single assessor and allowing the router to benefit from complementary evaluation perspectives.

4) Multi Candidate Generation and Deduplication: To improve the chances of generating a high-quality query for each chosen document, we use an LLM to produce multiple candidate queries and then pick the best one. For each selected document d_{i_t}, we sample n query candidates from the generation policy:

$$\mathcal{Q}_t = \{\, q_t^{(k)} \}_{k=1}^{n} \tag{14}$$

where $\hat{\mathbf{r}}_t^{(k)}$ is the normalized reward vector obtained if we were to use $q_t^{(k)}$. We then choose the highest-reward candidate among those that are not redundant with past queries:

$$q_t^\star \;=\; \arg\max_{q_t^{(k)} \in \mathcal{Q}_t} \; R_t^{(k)} \tag{15}$$

In other words, we compute the reward $R_t^{(k)}$ (Equatino 22) for each candidate query and pick the one with the largest $R_t^{(k)}$, while enforcing a cosine similarity threshold to ensure the chosen query $q_t^\star$ is not too similar to any previously generated query $q' \in Q_{1:t-1}$. This multi-candidate approach increases the likelihood of finding a strong query for each document and prevents the agent from generating slight rephrasings of earlier queries.

C. Reinforcement Learning for Mixture-of-Assessors

The MoA builds a cluster-aware evaluator that scores documents from complementary angles and fuses them adaptively to guide selection. Then reinforcement learning to generate a query for the chosen document, evaluates it and feeds the signal back through covariance based gating and light regularization. This closes the loop as clustering shapes assessor signals, the mixture selects documents, reinforcement learning measures downstream value, and the feedback adjusts weights and triggers occasional reclustering, yielding an adaptive and diversity preserving selection process.

1) Rewards Definition: When generate a query q_t for the selected document d_{i_t}, we evaluate the quality of q_t along four dimensions. To compute these, we embed the query q_t into the same vector space as the documents, so that we can measure similarities $s(x, y) = \cos(\mathbf{e}_x, \mathbf{e}_y)$ between any text x and y. Let $C_{c(i_t)}$ denote the cluster containing document d_{i_t}. We define the reward components $\mathbf{r}_t$.

a) Retrievability: This measures how well q_t retrieves its target document d_{i_t}. Let $r(q_t)$ be the rank position of d_{i_t} in the dense retriever's results for query q_t (where $r = 1$ means d_{i_t} is the top result). We convert this rank to a score:

$$r_{t,1} = \exp\Big(-\; \frac{r(q_t) - 1}{\tau_r} \Big) \tag{16}$$

where $\tau_r > 0$ is a temperature hyperparameter. This yields $r_{t,1} \approx 1$ if d_{i_t} is ranked near the top, and decays towards 0 as the rank position worsens. Thus, r_1 encourages queries that successfully retrieve their target documents.

b) Discriminativeness: A good query should be specific to the target document and not retrieve other documents from the same or similar clusters. Let d^{+}_{same} be the most similar other document to q_t within $C_{c(i_t)}$, aside from d_{i_t} itself, and let d^{+}_{near} be the top-ranked document retrieved from the most similar different cluster. $s_r(x, y)$ denotes the retriever-space similarity. We quantify query specificity as:

$$r_{t,2} = s_r(q_t,\, d_{i_t})\big(1 - s_r(q_t,\, d^{+}_{\text{same}})\big)\big(1 - s_r(q_t,\, d^{+}_{\text{near}})\big) \quad (17)$$

This reward is high if q_t is very similar to the target d_{i_t} while being dissimilar to the closest alternative document in the same cluster and to any document in other clusters. In essence, r_2 favors queries that distinguish d_{i_t} from other documents, making the retrieval task easier and more precise.

c) Answerability: We want the generated query to be answerable by the content of d_{i_t}. Let A_{i_t} be a representative answer-bearing span extracted from d_{i_t}. We define:

$$r_{t,3} = s_r(q_t,\, A_{i_t}) s_r(d_{i_t},\, A_{i_t})\Big(1 - \big|\, s_r(q_t, d_{i_t}) - z_r \,\big|\Big) \quad (18)$$

where $s(d_{i_t}, A_{i_t})$ is the similarity between the answer span and the document, and $z_r \in (0, 1)$ is a comfort zone parameter. The first two factors encourage q_t to be on-topic with d_{i_t} and its answer snippet, and the last factor penalizes q_t if its overall similarity to d_{i_t} is too high or too low relative to z. In practice, this helps prefer queries that are neither trivial nor completely unrelated, but rather ask about content that d_{i_t} can answer in a meaningful way.

d) Coverage: We reward queries that retrieve new relevant documents that were previously unselected. Let $\text{retrieved}(q_t)$ be the set of relevant documents among the results retrieved by q_t. We define the coverage gain as the fraction of currently unselected relevant documents that q_t manages to cover:

$$r_{t,4} = \frac{|\,\text{retrieved}(q_t) \cap U_t\,|}{|U_t|} \quad (19)$$

In other words, r_4 is high if q_t finds many new relevant documents that no previous query retrieved. This encourages the algorithm to broaden the coverage of the corpus. We also consider a variant of this metric that blends cluster-level and global coverage, those details here for brevity. To motivate our coverage reward r_4, we observe that it approximates the marginal gain of a near-submodular set function.

2) Covariance Based Assessor Reward Update: A naive approach to updating the assessor weights would be to tie each assessor directly to one reward component, increase w_j proportionally to $r_{t,j}$. However, such a hard binding of assessor to reward j can cause the policy to overfit to a single metric and collapse the assessor diversity. Instead, we derive an update direction for $\mathbf{w}$ from the empirical covariance between assessor scores and multi objective rewards. We maintain an exponential moving average estimate of the covariance between the assessor prior vector $\mathbf{S}_{i_t}$ and the normalized reward vector $\hat{\mathbf{r}}_t$. Let $\tilde{\mathbf{S}}_{i_t} = \mathbf{S}_{i_t} - \boldsymbol{\mu}_s$ and $\tilde{\mathbf{r}}_t = \hat{\mathbf{r}}_t - \boldsymbol{\mu}_r$ denote the mean centered assessor score vector and reward vector. $\boldsymbol{\mu}_s$ and $\boldsymbol{\mu}_r$ denote the means of the assessor score vector $\mathbf{S}_{i_t}$ and the normalized reward vector $\hat{\mathbf{r}}_t$, After each step, we update the covariance matrix as ($\mathbf{U} \in \mathbb{R}^{M\times m},\ \mathbf{V} \in \mathbb{R}^{T\times \ell},\ \boldsymbol{\Sigma}_{sr} \in \mathbb{R}^{m\times \ell}$):

$$\boldsymbol{\Sigma}_{sr} \;\leftarrow\; \rho\, \boldsymbol{\Sigma}_{sr} \;+\; (1-\rho)\Big(\mathbf{U}^{\top}\tilde{\mathbf{S}}_{i_t}\Big)\Big(\mathbf{V}^{\top}\tilde{\mathbf{r}}_t\Big)^{\top} \quad (20)$$

where $\rho \in [0, 1)$ is a smoothing factor (e.g., $\rho = 0.9$ retains a large fraction of past covariance). We then compute a gate update vector by projecting the current reward weight vector $\boldsymbol{\lambda}$ through this covariance matrix:

$$\mathbf{g}_t = \boldsymbol{\Sigma}_{sr}\, \boldsymbol{\lambda} \quad (21)$$

The vector $\mathbf{g}_t$ provides a "soft" association of assessors to the overall reward signal: each component $\mathbf{g}_{t,j}$ reflects how much assessor j score correlates with the current multi objective reward. This way, the update to $\mathbf{w}$ will consider how each assessor contributes to the blended objective, rather than tying it to a single reward component.

a) RL Based Global Coordination: Finally, we describe the RL updates that tie everything together. After executing query q_t, we have the normalized reward vector $\hat{\mathbf{r}}_t$ and aggregate reward $R_t = \boldsymbol{\lambda}^{\top}\hat{\mathbf{r}}_t$. We compute the advantage $A_t = R_t - b_t$ using a baseline b_t updated by an exponential moving average:

$$b_t \leftarrow (1 - \rho_b)\, b_{t-1} + \rho_b\, R_t \quad (22)$$

We then update the MoA gating weights $\mathbf{w}$ in the direction of $\mathbf{g}_t$ defined above, scaled by the advantage:

$$\mathbf{w} \;\leftarrow\; \Pi_{\Delta}\Big(\mathbf{w} \;+\; \eta_w\, A_t\, \mathbf{g}_t\Big) \quad (23)$$

Algorithm 1 Covariance Based Assessor Weight Update

Require: Current weights $W = [w_1, \ldots, w_M]$, Assessor scores $S_p = [s_1, \ldots, s_M]$ for selected document p, reward vector $r = [r_1, \ldots, r_M]$, baseline b, step index t, total steps T, step size η, hard alignment factor $\lambda_{\text{hard}}(t)$.

$R_{\text{total}} \leftarrow \frac{1}{M}\sum_{j=1}^{M} r_j$ ▷ total reward (mean of r_j)
$A \leftarrow R_{\text{total}} - b$ ▷ advantage (reward minus baseline)
$\Delta W_{\text{covar}} \leftarrow A \cdot \Big(S_p - \dfrac{1}{M}\sum_{j=1}^{M} s_j\Big)$ ▷ covariance update
$\Delta W_{\text{hard}} \leftarrow \big(r - \dfrac{1}{M}\sum_{j=1}^{M} r_j\big)$ ▷ centered reward
$\Delta W \leftarrow (1 - \lambda_{\text{hard}}(t))\, \Delta W_{\text{covar}} + \lambda_{\text{hard}}(t)\, \Delta W_{\text{hard}}$
$W \leftarrow W + \eta\, \Delta W$ ▷ gradient ascent step
if entropy regularization is enabled **then**
  $W \leftarrow W + \gamma\, \nabla H(W)$ ▷ add entropy gradient
end if
for $j = 1$ to M **do**
  $w_j \leftarrow \max(w_j,\ floorvalue)$ ▷ apply floor to each weight
end for
$W \leftarrow W / \sum_{j=1}^{M} w_j$ ▷ re-normalize weights to sum to 1
$b \leftarrow b + \beta \cdot (R_{\text{total}} - b)$ ▷ update baseline
return Updated weights W (for next selection)

Algorithm 2 *ReMoA* Pipeline

Require: Document corpus $D = \{d_i\}_{i=1}^{N}$ with embeddings $\{\mathbf{e}_i\}$, pretrained LLM generator, dense retriever $\mathcal{R}$
Initialize MoA gating weights $\mathbf{w}$
Initialize unselected sets $U \leftarrow D$ and $U^c \leftarrow C_c\ \forall c$, reward blend weights $\boldsymbol{\lambda} \leftarrow (0.25,0.25,0.25,0.25)$
Initialize moving baseline $b \leftarrow 0$ for policy gradient
for $t = 1$ **to** T **do**
  1 Assessor Selection Stage
  Compute initial cluster partition $\mathcal{C} = \{C_1, \ldots, C_K\}$ of D
  Choose cluster $c_t \leftarrow \arg\min_{c:|U^c|>0} |U^c|$
  Compute MoA scores $\phi(i) = \mathbf{w}^\top \mathbf{S}_i$ for each $i \in U^{c_t}$
  Select target document $i_t \leftarrow \arg\max_{i \in U^{c_t}} \phi(i)$
  2 Query Generation Stage
  Use LLM to generate n query candidates for d_{i_t}: $\{q_t^{(k)}\}_{k=1}^{n}$
  for each candidate $q_t^{(k)}$ **do**
    Retrieve top-K results with $\mathcal{R}(q_t^{(k)})$ and compute reward components $r_{t,1..M}^{(k)}$
    Normalize rewards $\hat{\mathbf{r}}_t^{(k)} \leftarrow \mathrm{Norm}(r_{t,1..M}^{(k)})$ and compute blended score $R_t^{(k)} = \boldsymbol{\lambda}^\top \hat{\mathbf{r}}_t^{(k)}$
  end for
  Select query $q_t^\star = \arg\max_{q_t^{(k)} Q_{1:t-1}} R_t^{(k)}$
  Add $q_t^\star$ to output set Q; mark d_{i_t} as covered ($U \leftarrow U \setminus \{d_{i_t}\}$, $U^{c_t} \leftarrow U^{c_t} \setminus \{d_{i_t}\}$)
  3 ReMoA Gating and Reward Update Stage
  Compute combined reward $R_t = \boldsymbol{\lambda}^\top \hat{\mathbf{r}}_t$ for chosen $q_t^\star$ and advantage $A_t = R_t - b$
  Update baseline $b \leftarrow (1-\rho_b)b + \rho_b R_t$
  Update covariance tracker: $\boldsymbol{\Sigma}_{sr} \leftarrow \rho\, \boldsymbol{\Sigma}_{sr} + (1-\rho)\, \tilde{\mathbf{S}}_{i_t} \tilde{\mathbf{r}}_t^\top$
  Compute gate gradient direction $\mathbf{g} = \boldsymbol{\Sigma}_{sr}\, \boldsymbol{\lambda}$
  Update gating weights: $\mathbf{w} \leftarrow \Pi_{\Delta,\epsilon}\big(\mathbf{w} + \eta_w A_t\, \mathbf{g}\big)$
  Update reward blend via rl: $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \eta_\lambda A_t(\hat{\mathbf{r}}_t - R_t \mathbf{1})$; $\boldsymbol{\lambda} \leftarrow \mathrm{softmax}(\boldsymbol{\beta})$
  Entropy-regularize gating: $\mathbf{w} \leftarrow (1-\gamma)\mathbf{w} + \gamma$; then floor $w_j \geq \epsilon$ for all j
  if drift in unselected set (low silhouette score) **then**
    Refresh clusters $\mathcal{C} \leftarrow \mathrm{Recluster}(U)$ and update U^c for each cluster c
  end if
end for
return Final query set $Q = \{q_1^\star, q_2^\star, \ldots, q_T^\star\}$

where η_w is a learning rate and Π_Δ projects the updated weights back onto the simplex. This is a form of policy gradient for the document selection policy, using $\mathbf{g}_t$ as a direction that increases expected reward.

Simultaneously, we update the reward blending weights $\boldsymbol{\lambda}$ via a policy-gradient step for multi-objective reward combination:

$$\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \eta_\lambda A_t \left(\hat{\mathbf{r}}_t - R_t\right) \tag{24}$$

where $\boldsymbol{\lambda}$ is the learning rate for $\boldsymbol{\lambda}$. The term $(\hat{\mathbf{r}}_t - R_t)$ is the reinforce policy gradient for a softmax selection (it has zero mean under the current blend, ensuring an unbiased update). Intuitively, if a particular component $\hat{r}_{t,j}$ is higher than the overall reward R_t, then $\boldsymbol{\lambda}_j$ will increase, whereas components that underperform the total reward will have their weights decreased.

To stabilize training in the early stages, we also implement a short *warm-up* for the gating weights. During the first few steps, we interpolate $\mathbf{w}$ towards the uniform distribution. After this period, we revert to the regular entropy smoothing and floor updates to continue preventing collapse.

D. Summary

Let I be the number of clustering iterations; let n be the number of query candidates, L the retrieval cutoff, $\mathsf{cost}_{\mathrm{ret}}(L)$ the per–candidate retrieval cost, and $\mathsf{cost}_{\mathrm{LLM}}$ the per–candidate generation cost. The complexity for selecting the target document d_{i_t} bundles clustering and assessor computation: building or refreshing $\mathcal{C}$ on U_t costs $O(|U_t|\, K\, M\, I)$. Generating and evaluating the query uses n candidates and top–L retrieval, costing $\mathsf{cost}_{\mathrm{gen+ret}}(L) := O\big(n(\mathsf{cost}_{\mathrm{LLM}} + \mathsf{cost}_{\mathrm{ret}}(L) + L)\big)$. Putting these together, one step simplifies to

$$O\Big(|U_t|\, KMI \;+\; n\, \mathsf{cost}_{\mathrm{gen+ret}}(L) I\Big),$$

Although this is higher than global top–k or single–assessor baselines, the gains in selection quality, long–tail coverage, and stability compensate for the added cost.

IV. Experiments

A. Setup

Datasets and Metrics. We evaluate on the BEIR benchmark. The datasets span diverse domains and tasks, including fact-checking, open-domain QA, paraphrase retrieval, and others, providing a robust zero-shot evaluation testbed. We report standard ranking metrics: nDCG@10 for re-ranking effectiveness and Recall@100 (R@100) for first-stage retrieval.

Baselines and Models. We compare against state-of-the-art unsupervised domain adaptation pipelines for IR, including **InPars** [38], **DocGen-RL** [39], and **Promptagator++**. InPars and DocGen-RL use LLMs to create synthetic supervision, while Promptagator++ is a strong pipeline employing a dense retriever and a LLM with consistency filtering. We also include conventional baselines: **BM25**, and **zero-shot** neural rankers (MonoT5-3B cross-encoder and ColBERT bi-encoder) without target-domain fine-tuning. Following [8], we fine-tune MonoT5-3B and ColBERT on synthetic query–document pairs produced by our method using auto-k or 1k generated queries.

Implementation Details. All experiments ran on a Slurm cluster (single node; 72 CPU cores; 463 GB RAM; 2×NVIDIA H100 80GB; Ubuntu 22.04; CUDA 12.2). We used Anaconda for environment management and PyTorch for training. Jobs were submitted via a `submit.sh` Slurm script that activates environments and requests resources. Each MonoT5-3B run uses one H100. We generate auto-k or 1k synthetic queries, fine-tune ColBERT for first-stage retrieval and MonoT5-3B for re-ranking on the same synthetic supervision.

B. Results and Discussion

Re-ranking Performance. Table I reports nDCG@10 across BEIR datasets. ***ReMoA*** fine-tuned re-rankers (MonoT5-3B) achieve the highest nDCG@10 on the majority of datasets, outperforming prior baselines. With 1k synthetic queries, ***ReMoA*** yields substantial gains over **InPars** and **DocGen-RL**, and improves upon the strong **Promptagator++** baseline

TABLE I: Main results (nDCG@10) on selected BEIR datasets with BM25 Top-100 candidates and a MonoT5-3B re-ranker. We compare *ReMoA* with baselines. The best score in each column is highlighted in **bold**. "DUQGen-reranker (1k/5k)" denotes MonoT5-3B fine-tuned with 1k/5k synthetic pairs; "*ReMoA* (Auto-k)" adaptively determines the query budget, while "*ReMoA* (1k)" uses a fixed 1k budget. The Avg column is computed over the datasets shown. Statistical significance follows the original convention: two-tailed paired t-test with Bonferroni correction ($p < 0.05$).

Model (↓) Datasets (→)	arguana	dbpedia-entity	fiqa	nfcorpus	nq	quora	scidocs	scifact	trec-covid	web-touche2020	Avg
Zero-shot MonoT5-3B	0.316	0.408	0.462	0.373	0.579	0.848	0.193	0.760	0.830	0.311	0.508
InPars	–	0.315	0.352	–	0.313	–	–	–	0.803	–	–
DocGen-RL	–	–	–	–	0.517	–	–	–	–	–	–
DUQGen-reranker(1k)	0.393	0.422	0.458	0.382	0.593	0.895	0.200	0.757	0.862	**0.320**	0.528
DUQGen-reranker(5k)	**0.427**	0.421	0.465	0.376	0.588	0.894	0.202	0.761	0.836	0.269	0.524
***ReMoA* (Auto-k)**	0.419	**0.433**	0.467	**0.384**	**0.603**	0.909	**0.205**	**0.775**	**0.865**	0.318	**0.538**
***ReMoA* (1k)**	0.416	0.432	**0.473**	0.379	0.593	**0.910**	0.204	0.769	0.839	0.315	0.533

TABLE II: Retriever-only performance (Recall@100) on selected BEIR datasets. We compare BM25, Zero-shot ColBERT, DUQGen-retriever, and *ReMoA*-retriever. The best score in each column is in **bold**. *ReMoA*-retriever attains the highest average Recall@100. These results indicate that the RL-collaborative MoA selection yields broader coverage and stronger first-stage retrieval, providing better candidates for downstream re-ranking.

Models (↓) Datasets (→)	arguana	dbpedia-entity	fiqa	nfcorpus	nq	quora	scidocs	scifact	trec-covid	web-touche2020	Avg
BM25	**0.942**	0.398	0.540	0.250	0.760	0.973	0.356	0.908	0.498	**0.538**	0.616
Zero-shot ColBERT	0.885	0.458	0.598	0.255	0.911	0.989	0.345	0.878	0.473	0.436	0.623
DUQGen-retriever	0.906	**0.493**	0.615	0.272	0.915	0.990	0.356	0.899	0.544	0.474	0.646
MoA-QGEN-retriever	0.915	0.492	**0.625**	**0.278**	**0.923**	**0.991**	**0.362**	**0.909**	**0.558**	0.490	**0.654**

on shared datasets, despite using far fewer queries (1k vs. millions). On small paraphrase-style tasks, Promptagator++ is occasionally competitive; ***ReMoA*** dominates elsewhere. Statistical significance ($p < 0.05$) is marked where applicable.

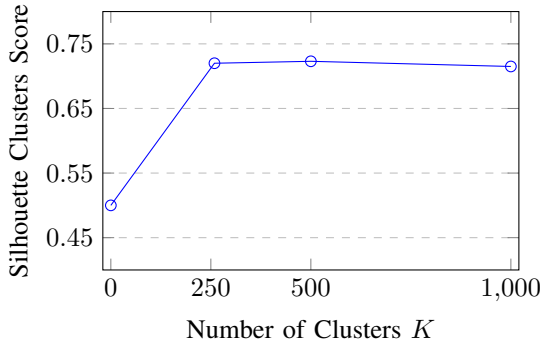


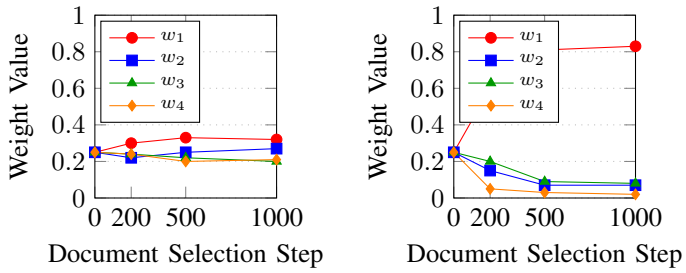
Fig. 3: Silhouette coefficient vs. cluster count k for an example input. Here $k = 259$ maximizes the score, which Auto-K selects. Results are based on the SCIFACT dataset.

Table III shows fine-tuning MonoT5-3B on synthetic (q, p) pairs shows that the adaptive controller auto-k attains the highest nDCG@10 on datasets, outperforming fixed 1k/5k budgets by selecting representative documents without under/over-generation. In Table IV,the ablation isolates the effect of RL in *ReMoA*: comparing a MoA-only router against MoA+RL under fixed budgets (1k/5k) and the adaptive auto-k, RL consistently boosts nDCG@10, with the strongest gains when combined with auto-k, confirming the value of RL-controlled reward weighting and assessor reweighting.

TABLE III: Ranking performance (nDCG@10) on development sets with different training sizes. We fine-tune a MonoT5-3B re-ranker on synthetic (q, p) pairs produced by *ReMoA*; N denotes the query budget and "Auto-k" is our adaptive controller. The best score per column is in **bold**. Auto-k achieves the highest nDCG@10 on both SCIFACT and NQ, outperforming fixed 1k/5k budgets. This indicates that the RL-coordinated MoA can allocate an appropriate number of queries per corpus without manual tuning, improving generalization while avoiding under/over-generation.

Model	N	Scifact	NQ
MonoT5-3B	0	0.7600	0.5792
MonoT5-3B	Auto-k	**0.7752**	**0.6033**
MonoT5-3B	1k	0.7693	0.5933
MonoT5-3B	5k	0.7699	0.5945

The Fig. 3 shows silhouette-based cluster analysis further shows that *ReMoA* picks the optimal cluster count (e.g. $k = 259$ for SCIFACT) that balances cohesion and separation. Overall, *ReMoA* consistently attains higher nDCG and Recall with significantly fewer queries, leading to higher score at a lower query budget. Fig. 4 illustrates the trajectories of assessor weights across query-generation steps in two settings: (a) with our proposed covariance-based regularization and entropy smoothing, and (b) without any regularization. In the regularized setting, the weights w remain relatively balanced, ensuring that no single assessor dominates the generation process. In contrast, without regularization, one assessor here, w, corresponding to the similarity density assessor quickly dominates after the first step, eventually absorbing nearly the



(a) With Regularization (b) No Regularization

Fig. 4: Assessor weight trajectories over document selection steps for (a) *ReMoA* with covariance + entropy regularization, and (b) without any regularization. With regularization, weights remain balanced; without, one assessor (red, assessor NI) dominates by the end, suppressing others.

entire weight mass. This collapse suppresses the diversity of assessor guidance and limits the variety of generated queries. The figure demonstrates the effectiveness of our regularization strategy in preserving assessor balance and preventing mode collapse.

TABLE IV: Ablation on Auto-k: effect of RL in *ReMoA* (nDCG@10). We compare a MoA-only router with MoA+RL under fixed query budgets (1k/5k) and the adaptive Auto-k setting. The best score per column is in **bold**. Adding RL consistently improves across budgets, and coupling RL with Auto-k yields the strongest results. These gains indicate that learning to reweight Assessors and rewards, together with an adaptive query budget, best aligns query generation with downstream re-ranking.

Model	N	Scifact	NQ
ReMoA (MoA)	Auto-k	0.7695	0.5954
ReMoA (MoA)	1k	0.7631	0.5951
ReMoA (MoA)	5k	0.7645	0.5960
ReMoA (MoA + RL)	Auto-k	**0.7752**	**0.6033**
ReMoA (MoA + RL)	1K	0.7672	0.5933
ReMoA (MoA + RL)	5K	0.7699	0.5945

Our experiments demonstrate that *ReMoA* effectively coordinates multiple Assessors and reward signals to generate high-quality query sets that improve retrieval recall without sacrificing precision. One key strength is the ability to automatically adjust the number of queries to the complexity of the information need. Traditional re-rankers strategies would require either guessing the number of queries or always using a fixed number. *ReMoA*'s stopping criterion (cover most features or reach K) elegantly handles this. *ReMoA* achieves higher score on benchmarks, despite using far fewer synthetic queries.

At the first-stage retrieval level, *ReMoA* also outperforms mainstream baselines. As shown in Table II, the *ReMoA* retriever attains an average Recall@100 of 0.654, significantly above BM25's 0.616. This indicates that *ReMoA* 's query selection covers a broader set of relevant documents, providing stronger candidates for re-ranking.

And important advantage is efficiency: *ReMoA* achieves these gains with far lower query cost. *ReMoA* dynamically adjusts the number of queries to the complexity of each input, stopping when coverage is achieved. In practice, this means trivial cases use minimal queries and complex cases receive only as many as needed.

One limitation of our current evaluation is the reliance on known relevant documents to compute rewards r_4 and measure coverage. In a fully unsupervised setting where gold relevant sets are not known, one cannot directly calculate coverage. However, *ReMoA* could still operate in a pseudo-relevance feedback manner: treat top results as provisionally relevant and encourage finding new ones. The submodular nature of coverage would still hold for novel document discovery. We leave exploration of an unsupervised variant for future work.

Finally, while BEIR is a focused benchmark, the techniques in *ReMoA* are generally applicable to other domains where multi-query strategies and diversity matter (web search sessions, multi-hop QA, etc.). The modular design means one can plug in different Assessor heuristics or reward definitions for a given domain, and the RL will adjust accordingly. For example, in a product search scenario, Assessors might encode price, popularity, and rewards could include user click signals. We believe *ReMoA* opens up a flexible framework for such settings.

V. Conclusion and Future Work

We presented *ReMoA*, a reinforcement-guided Mixture-of-Assessors framework for domain adaptive query generation. At its core, a lightweight router aggregates scores from multiple assessors to select representative documents, while the generator conditions on the selected document to produce diverse pseudo-queries with semantic de-duplication. Retrieval-grounded, multi-objective feedback coordinates routing and generation through covariance regularized updates, and a quota-based cluster policy allocates selection budget across topics to maintain coverage without over-focusing on easy regions. Together, these components form a coherent control loop that prioritizes representative D and yields a diversified Q aligned with downstream retrieval. Empirically, *ReMoA* delivers consistent gains over strong query-generation pipelines across multiple target domains, improving both first-stage recall and re-ranking effectiveness, with ablations confirming the contribution of MoA routing, multi-objective feedback, and coverage-aware selection.

In future work, we will reduce the per–step cost and broaden *ReMoA*'s applicability along several axes. First, replace explicit neighbor lists in the similarity–density prior with an approximate nearest–neighbor index, yielding sublinear candidate search and lowering the effective factors in K and d . Second, compress document/query embeddings and compute in the compressed space to shrink the d factor throughout clustering and scoring. Third, make evaluation streaming: emit retrieval results as soon as rewards are computed, and track coverage via bitsets/Bloom filters instead of caching top-L

sets, reducing both the L term and memory pressure. Fourth, adapt n and L online from reward uncertainty, allowing early stopping when a dominant candidate emerges, thus lowering the $n\,\mathrm{cost}_{\mathrm{gen+ret}}(L)$ term. Fifth, replace full de-duplication with sketching to avoid linear growth in past-query comparisons. Sixth, simplify routing by distilling the MoA router to a lightweight scorer or by using sparse top-2 gating, cutting mixture overhead while preserving diversity. Seventh, amortize cluster refresh with mini-batch/online k-means and approximate silhouette triggers, reducing the I and $|U_t|$ impact. Finally, joint training of the generator and retriever enables shared encoders and indices, eliminating redundant passes and further reducing both compute and memory. These steps together aim to turn the dominant factors in $|U_t|, K, d, I, n, L$ into smaller *effective* constants, improving wall-clock efficiency and scaling *ReMoA* to larger corpora and domains.

References

[1] B. Mitra, N. Craswell *et al.*, "An introduction to neural information retrieval," *Foundations and Trends® in Information Retrieval*, vol. 13, no. 1, pp. 1–126, 2018.

[2] A. Yates, R. Nogueira, and J. Lin, "Pretrained transformers for text ranking: BERT and beyond," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Tutorials*. Association for Computational Linguistics, Jun. 2021, pp. 1–4.

[3] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models," *arXiv preprint arXiv:2104.08663*, 2021.

[4] T. Shen, G. Long, X. Geng, C. Tao, Y. Lei, T. Zhou, M. Blumenstein, and D. Jiang, "Retrieval-augmented retrieval: Large language models are strong zero-shot retriever," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 15 933–15 946.

[5] M. Gospodinov, S. MacAvaney, and C. Macdonald, "Doc2query–: when less is more," in *European Conference on Information Retrieval*. Springer, 2023, pp. 414–422.

[6] X. Li, J. Jin, Y. Zhou, Y. Zhang, P. Zhang, Y. Zhu, and Z. Dou, "From matching to generation: A survey on generative information retrieval," *ACM Transactions on Information Systems*, vol. 43, no. 3, pp. 1–62, 2025.

[7] O. Young, Y. Fan, R. Zhang, J. Guo, M. de Rijke, and X. Cheng, "Gaqr: An efficient generation-augmented question rewriter," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 4228–4232.

[8] R. Chandradevan, K. Dhole, and E. Agichtein, "DUQGen: Effective unsupervised domain adaptation of neural rankers by diversifying synthetic query generation," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, Jun. 2024, pp. 7437–7451.

[9] K. D. Dhole and E. Agichtein, "Genqrensemble: Zero-shot llm ensemble prompting fornbsp;generative query reformulation," in *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, 2024, p. 326–335.

[10] K. Sawarkar, A. Mangal, and S. R. Solanki, "Blended rag: Improving rag (retriever-augmented generation) accuracy with semantic search and hybrid query-based retrievers," in *2024 IEEE 7th international conference on multimedia information processing and retrieval (MIPR)*. IEEE, 2024, pp. 155–161.

[11] A. Sinha, P. R. Mall, and D. Roy, "Exploring the nexus between retrievability and query generation strategies," in *European Conference on Information Retrieval*. Springer, 2024, pp. 177–193.

[12] D. Y. Hwang, B. Taha, H. Pande, and Y. Nechaev, "Link, synthesize, retrieve: Universal document linking for zero-shot information retrieval," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 18 971–18 982.

[13] S. Kang, B. Jin, W. Kweon, Y. Zhang, D. Lee, J. Han, and H. Yu, "Improving scientific document retrieval with concept coverage-based query set generation," in *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*, 2025, pp. 895–904.

[14] A. Chaudhary, K. Raman, and M. Bendersky, "It's all relative! – a synthetic query generation approach for improving zero-shot relevance prediction," in *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 1645–1664.

[15] T. Almeida and S. Matos, "Exploring efficient zero-shot synthetic dataset generation for information retrieval," in *Findings of the Association for Computational Linguistics: EACL 2024*, 2024, pp. 1214–1231.

[16] D. Kim, Y. Jang, D. Shin, C. Park, and Y. Ko, "Hyper-QKSG: Framework for automating query generation and knowledge-snippet extraction from tables and lists," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, US: Association for Computational Linguistics, Nov. 2024, pp. 1351–1360.

[17] J. Guo, Y. Cai, K. Bi, Y. Fan, W. Chen, R. Zhang, and X. Cheng, "Came: Competitively learning a mixture-of-experts model for first-stage retrieval," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–25, 2025.

[18] H. Lee, L. Soldaini, A. Cohan, M. Seo, and K. Lo, "Routerretriever: Routing over a mixture of expert embedding models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 11, 2025, pp. 11 995–12 003.

[19] Y. Li, H. Zhang, Y. Zhang, X. Ma, W. Ye, N. Song, S. Wang, H. Xiong, D. Yin, and L. Chen, "M 2 oerank: Multi-objective mixture-of-experts enhanced ranking for satisfaction-oriented web search," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 4441–4454.

[20] Q. Dong, Y. Liu, Q. Ai, Z. Wu, H. Li, Y. Liu, S. Wang, D. Yin, and S. Ma, "Unsupervised large language model alignment for information retrieval via contrastive feedback," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 48–58.

[21] Y. Jiang, S. Zhao, J. Li, H. Wang, and B. Qin, "GainRAG: Preference alignment in retrieval-augmented generation through gain signal synthesis," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 10 746–10 757.

[22] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 39–48.

[23] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," *arXiv preprint arXiv:2112.09118*, 2021.

[24] J. Ni, C. Qu, J. Lu, Z. Dai, G. Hernandez Abrego, J. Ma, V. Zhao, Y. Luan, K. Hall, M.-W. Chang, and Y. Yang, "Large dual encoders are generalizable retrievers," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 9844–9855.

[25] R. Nogueira, W. Yang, K. Cho, and J. Lin, "Multi-stage document ranking with bert," *arXiv preprint arXiv:1910.14424*, 2019.

[26] R. Nogueira, Z. Jiang, R. Pradeep, and J. Lin, "Document ranking with a pretrained sequence-to-sequence model," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Nov. 2020, pp. 708–718.

[27] P. Zhu and C. Hauff, "Unsupervised domain adaptation for question generation with DomainData selection and self-training," in *Findings of the Association for Computational Linguistics: NAACL 2022*. Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 2388–2401.

[28] M. Li, H. Zhuang, K. Hui, Z. Qin, J. Lin, R. Jagerman, X. Wang, and M. Bendersky, "Can query expansion improve generalization of strong cross-encoder rankers?" in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information*

Retrieval, ser. SIGIR '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2321–2326.

[29] Y. Wang, H. Zhang, L. Pang, B. Guo, H. Zheng, and Z. Zheng, "Maferw: Query rewriting with multi-aspect feedbacks for retrieval-augmented large language models," in *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, T. Walsh, J. Shah, and Z. Kolter, Eds. AAAI Press, 2025, pp. 25 434–25 442.

[30] B. Engelmann, T. Breuer, J. I. Friese, P. Schaer, and N. Fuhr, "Context-driven interactive query simulations based onnbsp;generative large language models," in *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2024, p. 173–188.

[31] T. Breuer, "Data fusion of synthetic query variants with generative large language models," in *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, 2024, pp. 274–279.

[32] Y. Lee, M. Kim, and S. Hwang, "Disentangling questions from query generation for task-adaptive retrieval," *CoRR*, vol. abs/2409.16570, 2024.

[33] M. R. Calvo, S. Ding, C. D. Barrett, G. Dinu, and G. Karypis, "Beyond instruction-conditioning, MoTE: Mixture of task experts for multi-task embedding models," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 22 731–22 746.

[34] Z. Su, X. Wu, Z. Lin, Y. Xiong, M. Lv, G. Ma, H. Chen, S. Hu, and G. Ding, "Cartesianmoe: Boosting knowledge sharing among experts via cartesian product routing in mixture-of-experts," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Association for Computational Linguistics, 2025, pp. 10 040–10 055.

[35] S. Hsu, O. Khattab, C. Finn, and A. Sharma, "Grounding by trying: Llms with reinforcement learning-enhanced retrieval," in *International Conference on Representation Learning*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, Eds., vol. 2025, 2025, pp. 26 136–26 152.

[36] S. Soleimany, S. Ebrahimi, S. Seyedsalehi, F. Zarrinkalam, and E. Bagheri, "Reinforcement learning for effective few-shot ranking," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 2956–2961.

[37] Z. Ke, W. Kong, C. Li, M. Zhang, Q. Mei, and M. Bendersky, "Bridging the preference gap between retrievers and llms," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, L. Ku, A. Martins, and V. Srikumar, Eds. Association for Computational Linguistics, 2024, pp. 10 438–10 451.

[38] L. Bonifacio, H. Abonizio, M. Fadaee, and R. Nogueira, "Inpars: Unsupervised dataset generation for information retrieval," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2387–2392.

[39] A. Askari, M. Aliannejadi, C. Meng, E. Kanoulas, and S. Verberne, "Expand, highlight, generate: RL-driven document generation for passage reranking," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 10 087–10 099.